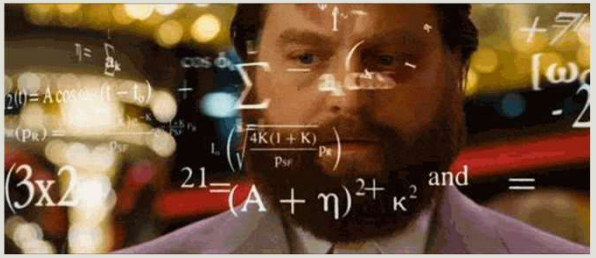
Angular La renaissance
GL4
TP1

AYMEN SELLAOUTI

Les règles du jeu

1. **Lorsque je vous demande de m'expliquer votre code et que vous le lisez a ce moment la c'est que ce n'est pas votre code et que vous ne l'avez pas lu ni compris donc je noterai celui qui l'a écrit et pour vous c'est un 0.**
2. **Vous devez toujours réfléchir avant de commencer a code ou a parler a l'IA.**
3. **Une architecture cohérente, un code propre, lisible et factorisé, l'application des designs patterns n'est pas une option c'est un devoir.**
4. **Considérer la validation comme un entretient, le jour J vous devez être prête et très bien connaitre votre sujet.**

Exercice

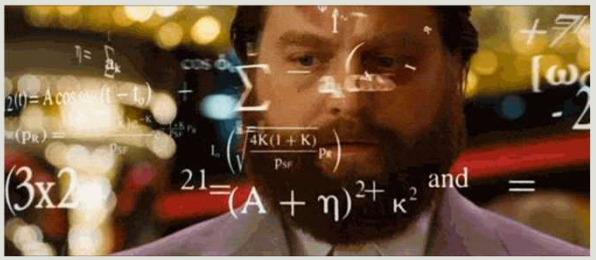


- Cloner le répo suivant :

https://github.com/aymensellaouti/gl4AngularRenaissanceTPStarting

- Ce répo est un projet ng18 mais qui est implémenté, en majorité, en utilisant les fonctionnalités d'angular 14, 15 et 16.
- Afin de migrer vers la version 18 à partir de la version 16, j'ai effectué deux migration : 16>17 puis 17>18 en utilisant la commande :
- ng update @angular/core@17 @angular/cli@17 puis
- ng update @angular/core@18 @angular/cli@18
- De votre coté, **créer une branche migration**, ensuite continuer le travail jusqu'à la version 22 puis vous allez utilisez les commandes vous permettant de :
  - Migrer vers les standalones components
  - Utilisez le nouveau control flow
  - Utilisez la fonction inject

Exercice



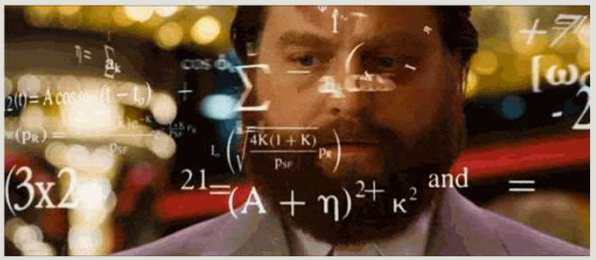
Angular offre un moyen **de migrer automatiquement une partie** de votre application Angular modulaire (15.2.0) et plus vers une application standalone.

- Vous pouvez suivre les différentes étapes du Guide qui présente des parties automatiques et d'autres manuelles.
- Exécutez la migration **dans l'ordre indiqué** ci-dessous, **en vérifiant que votre code est généré et exécuté entre chaque étape** :

1. Exécutez ng g @angular/core:standalone et sélectionnez "Convert all components, directives and pipes to standalone"
2. Exécutez ng g @angular/core:standalone et sélectionnez "Remove unnecessary NgModule classes" , **cette étape risque de garder plusieurs modules.**
3. Exécutez ng g @angular/core:standalone et sélectionnez "Bootstrap the project using standalone APIs".

https://angular.io/guide/standalone-migration

Exercice

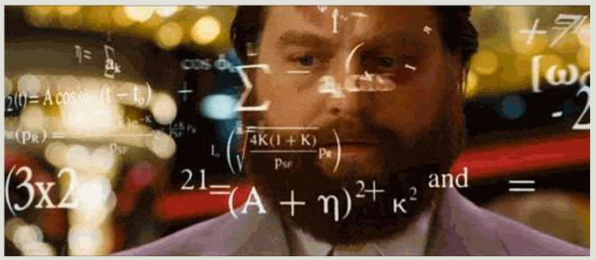


- Si vous voulez migrer votre ancien code et utilisez le nouveau work flow, angular vous fournie une commande qui le fait pour vous :
  - **ng g @angular/core:control-flow**

- Cette fonctionnalité est encore en developer Preview pour Anguar 17.2

https://angular.dev/reference/migrations/control-flow

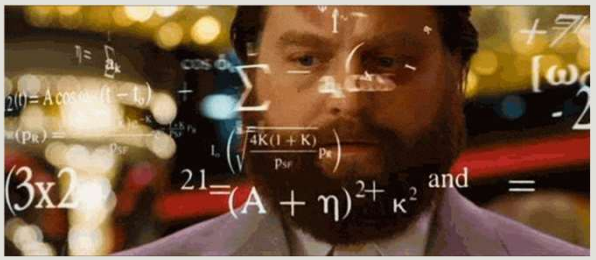
Exercice



- Afin d'utiliser la nouvelle fonction inject, vous pouvez utiliser la commande : **ng generate @angular/core:inject**

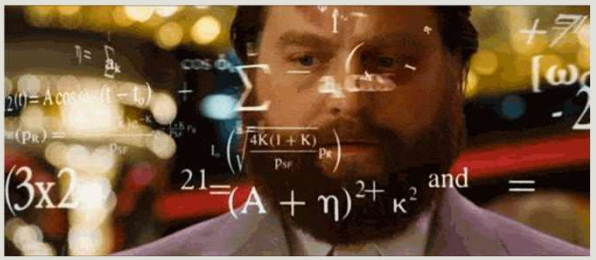
https://angular.dev/reference/migrations/inject-function

Exercice



- Pour les exercices suivants, créer deux branches, une appelé **rxjs**, **basée sur le projet initial**, l'autre appelé **signal basée sur la branche migration.**
- Pour les exercices sur RxJs et les fonctionnalités associées aux modules, utilisez la branche **rxjs**, pour les exercices pour les signaux, utilisez la branche **signal**

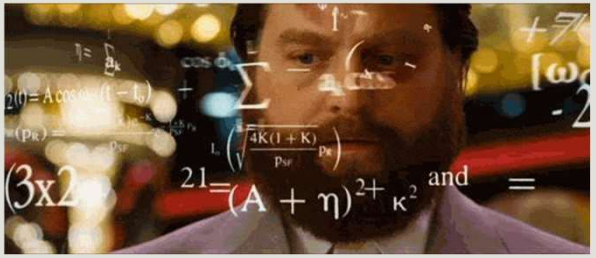
Exercice

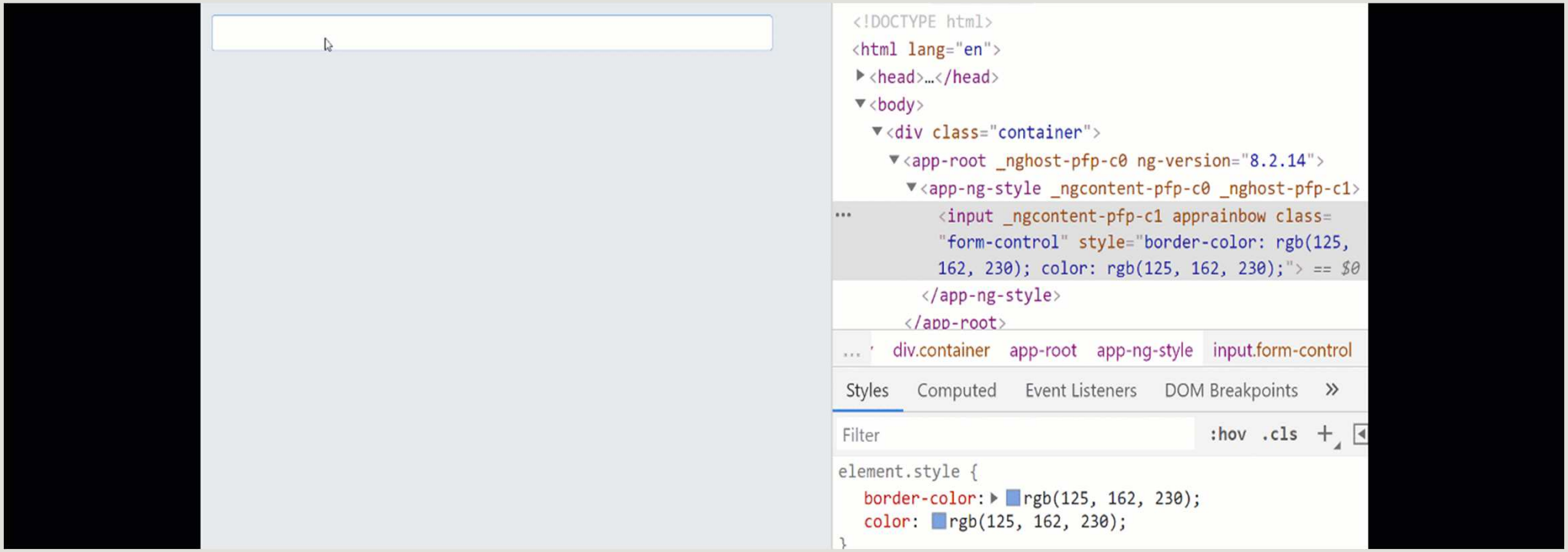


Dans **les deux branches** on va créer un truc plus sympas on va créer un simulateur d'écriture arc en ciel. **Faites en sorte que dans la branche signal on utilise les signaux pour voir la différence.**

- Créer une directive
- Créer un hostbinding sur la couleur et la couleur de la bordure.
- Créer une méthode qui générer une couleur hexadécimale aléatoirement.
- Faite en sorte qu'en appliquant votre directive à un input, à chaque écriture d'une lettre (event keyup) la couleur change en prenant aléatoirement une couleur.
- **Il faut que la directive soit instanciée uniquement pour les éléments auxquels elle peut s'appliquer. Exemple, cette directive n'a aucun impact sur une DIV.**

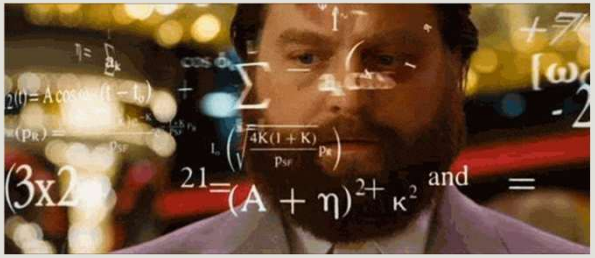
Exercice


(3x2
21=(A + η)²+ κ²
and


<!DOCTYPE html>
<html lang="en">
<head>…</head>
<body>

<app-root _nghost-pfp-c0 ng-version="8.2.14">
<app-ng-style _ngcontent-pfp-c0 _nghost-pfp-c1>
<input _ngcontent-pfp-c1 apprainbow class="form-control" style="border-color: rgb(125, 162, 230); color: rgb(125, 162, 230);"> == $0
</app-ng-style>
</app-root>
div.container app-root app-ng-style input.form-control
Styles Computed Event Listeners DOM Breakpoints
Filter :hov .cls
element.style {
border-color: rgb(125, 162, 230);
color: rgb(125, 162, 230);
}

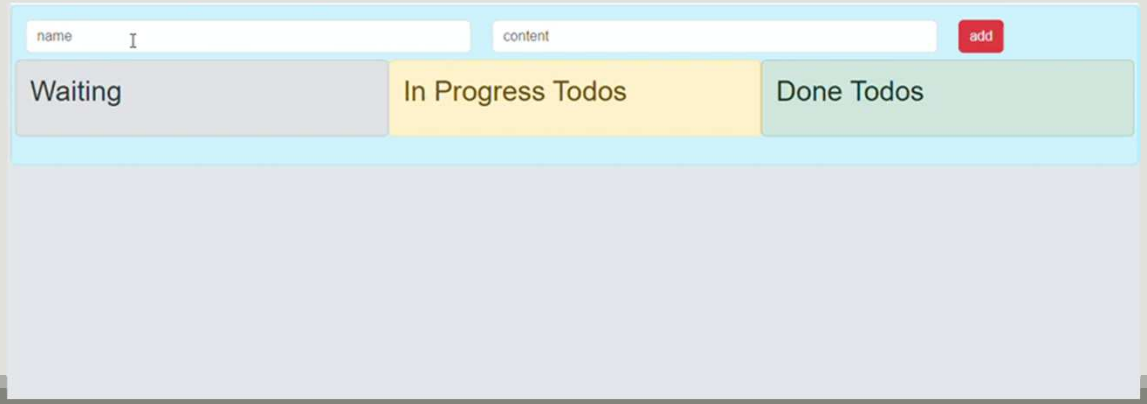
Exercice



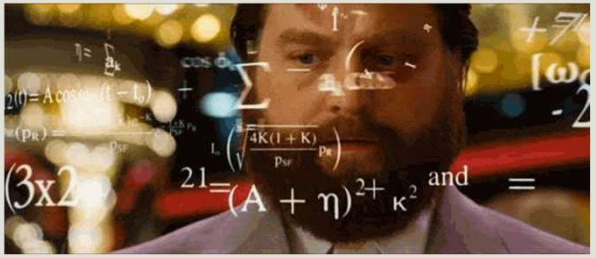
- Reproduisez cette interface en utilisant les signaux.
- Un Todo possède un id, name, content et un statut.
- Le statut est un type définit

```
export type TodoStatus =  'waiting' | 'in progress' | 'done';
```

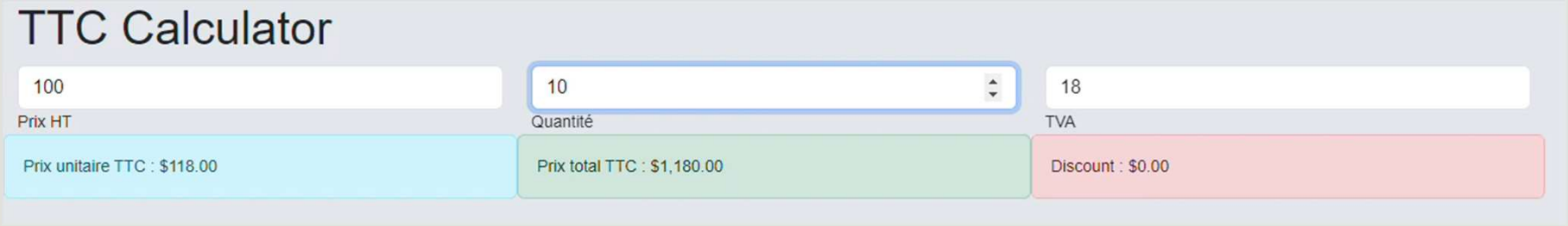
- A chaque changement de statut, l'élément va automatiquement changer de liste.
- Définissez votre composant et service afin de réaliser ce travail



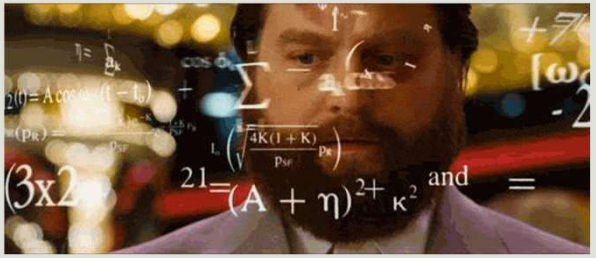
Exercice



- **Dans la branche signal,** créer un composant TTC qui permet de calculer le prix TTC d'un produit selon le nombre de pièces et la TVA. Sachant que l'utilisateur peut changer toutes les valeurs et que par défaut la quantité est de 1 le prix est de 0 et la tva est de 18.
- Si le nombre de pièces est entre 10 et 15 une remise de 20% est appliquée.
- Si le nombre de pièces est supérieur à 15 une remise de 30% est appliquée.

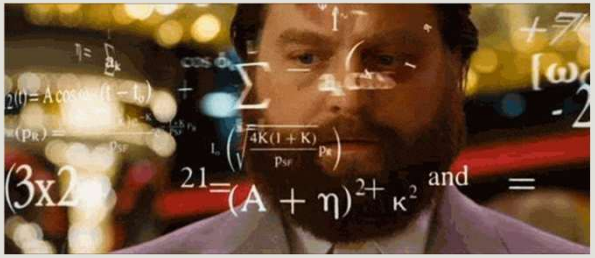


Exercice



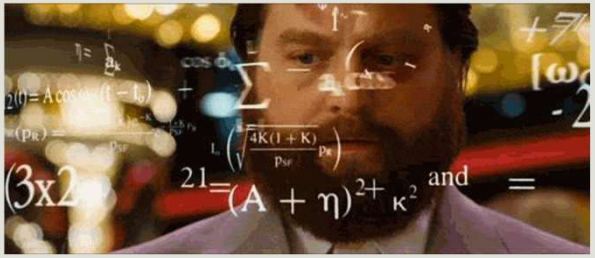
- Dans la branche signal, reprenez la cvTech (ce qui se trouve dans le dossier Cv) et utilisez les signaux quand c'est nécessaire.
- Réfléchissez à l'architecture du signal

Exercice



- La partie authentification est actuellement gérée d'une façon standard. Analyser le dossier auth ainsi que la navbar et le detailsComponent.
- Nous voulons utiliser les signaux afin d'avoir un code optimisé et réactif.
- Nous voulons aussi garder la trace du user authentifié en sauvegardant son id et son email, et de savoir qu'il est authentifié ou non.
- Au chargement de l'application vous devez recharger l'état de l'utilisateur.
- **N'utilisez pas de state management library**

Exercice



Dans la branche rxjs,

- Reprenez les composants CvComponent et DetailsCvComponent.
- Utilisez l'async pipe à chaque fois que vous le pouvez.
- Utilisez catchError pour gérer les erreurs.